Centro Universitário UniFECAF

Bárbara Maria Rodrigues Leão

ENTREGA CONTÍNUA DA PLATAFORMA DE PEDIDOS EM MICROSSERVIÇOS

São Paulo – SP

2026

PARTE TEÓRICA

INTRODUÇÃO

A Loja Veloz precisa se preparar para uma campanha em que mais pessoas vão comprar ao mesmo tempo, contudo no atual cenário, uma atualização pode interromper o site.

Assim, quando uma compra demora, a equipe também tem dificuldade para descobrir se o problema está no estoque, no pagamento ou no próprio atendimento ao cliente. Para resolver essas questões, neste trabalho, a compra de um mouse serve de exemplo para acompanhar essas etapas e verificar o que acontece quando o sistema recebe mais acessos.

ORGANIZAÇÃO DOS SERVIÇOS

Para o cliente, a compra é uma só, na loja, ela envolve quatro serviços, sendo o primeiro, chamado API Gateway, onde recebe a solicitação, depois o serviço de pedidos coordena as etapas e tem o de estoque que verifica se o produto está disponível e reserva a quantidade solicitada. E por fim o pagamento devolve o resultado da autorização.

Quando o pagamento é aprovado, a compra fica registrada no PostgreSQL e a reserva é baixada. Essa divisão é a base da arquitetura de microsserviços, onde cada serviço tem uma responsabilidade e pode receber uma atualização própria, desde que continue a trocar informações no formato que os demais esperam.

A divisão também traz uma dificuldade, pois se a resposta do pagamento se perder, repetir a cobrança pode prejudicar o cliente, e, para trabalhar com uma operadora real, a loja precisaria identificar cada tentativa e consultar seu resultado antes de cobrar novamente.

O PAPEL DO DEVOPS

Na Loja Veloz, o trabalho de quem desenvolve uma mudança continua depois que ela chega ao site. Antes de publicar, a equipe registra o código e as configurações no repositório, testa a compra e identifica a versão que será usada. Depois, acompanha os pedidos junto com quem cuida da operação, o que facilita descobrir qual mudança precisa ser revista se os clientes começarem a ter dificuldades.

Uma referência para essa organização foi a Online Boutique, aplicação de

exemplo do Google Cloud em que o serviço de checkout coordena etapas como pagamento e entrega. Na Loja Veloz, essa função ficou com o serviço de Pedidos. Assim, o Gateway recebe a solicitação do cliente e Pedidos acompanha o que ainda falta para concluir a compra.

CONTAINERS E ATUALIZAÇÕES

Para que outra pessoa consiga executar o projeto com as mesmas bibliotecas, o Docker reúne a aplicação e os arquivos de que ela precisa em uma imagem. A partir dessa imagem é possível iniciar um container. Como a loja depende de vários serviços ao mesmo tempo, o Docker Compose descreve como eles se conectam e quais dados precisam ser guardados, permitindo iniciar todo o ambiente com um comando.

Com a loja em funcionamento, o Kubernetes mantém cópias dos serviços disponíveis e encaminha as solicitações para aquelas que estão prontas para atender. A integração e a entrega contínuas, conhecidas pela sigla CI/CD, organizam as verificações entre a alteração do código e a publicação de uma versão. Se o teste de estoque indicar que a última unidade pode ser vendida duas vezes, por exemplo, a publicação deve parar para que a equipe corrija a falha. Como duas versões podem atender clientes durante a troca, qualquer mudança nas tabelas do banco também precisa funcionar com a versão anterior.

Para o desenvolvimento e os testes, escolhi o Docker Compose porque ele permite iniciar todos os serviços da loja em uma única máquina com um só comando. Já em produção, a loja precisa continuar recebendo pedidos durante uma atualização ou um pico de acessos. Por isso, escolhi o Kubernetes, que permite distribuir as cópias dos serviços entre várias máquinas, substituir as que falham, aumentar a quantidade de cópias conforme a demanda e controlar quais serviços podem se comunicar entre si.

COMO ACOMPANHAR UMA COMPRA

Quando um cliente relata que o pagamento demorou, as métricas ajudam a verificar se o tempo de resposta aumentou para outras pessoas. Para investigar aquela compra em particular, a equipe pode abrir seu rastreamento no Jaeger e

comparar o tempo gasto em Estoque e em Pagamentos. Com o identificador da compra, também consegue localizar nos logs o que cada serviço registrou naquele momento.

Depois de confirmar a compra, a loja precisa avisar os outros serviços de que há um novo pedido. Por isso, grava o pedido e o aviso na mesma operação no banco, de modo que a mensagem fique guardada mesmo se o Redis estiver fora do ar. Quando a conexão volta, o envio pode ser retomado. Caso a conexão caia logo depois do envio, quem recebe a mensagem precisa conferir seu identificador para evitar que o mesmo aviso gere duas ações.

DECISÕES DO PROJETO

Como a loja tinha quatro semanas para se preparar para a campanha, escolhi o RollingUpdate: a cópia antiga continua atendendo até que a nova esteja pronta, o que exige espaço para apenas uma cópia adicional por vez. O blue/green exigiria manter dois ambientes completos durante a troca, e o canary dependeria de uma ferramenta como o Argo Rollouts e de critérios para ampliar o acesso à versão nova. Por isso, o canary ficou para uma próxima etapa.

Para lidar com o pico de acessos, o HPA aumenta as cópias do Gateway e de Pedidos conforme o uso de CPU e, em Pedidos, também de memória. Em Estoque e Pagamentos, o VPA apenas recomenda quanta CPU e memória reservar, para que os dois mecanismos não ajustem a mesma métrica de um serviço. E no aviso de novo pedido fica guardado em uma tabela de saída no PostgreSQL e é enviado pelo Redis Streams, o que é suficiente para a primeira versão. O Kafka faria mais sentido se vários serviços precisassem processar esses avisos de forma independente.

E para acompanhar cada compra, o OpenTelemetry registra as etapas e o Jaeger mostra o tempo gasto em cada serviço. Uma service mesh como o Istio pode ser avaliada quando a loja precisar de mais controle sobre o tráfego e a autenticação entre os serviços. Por fim, descrevi a infraestrutura prevista para a AWS em Terraform, para que cada alteração seja revisada, tenha histórico e possa ser usada nos ambientes de teste e de produção.

PARTE PRÁTICA

AMBIENTE DE DESENVOLVIMENTO

Para executar o projeto, o ponto de partida é o README. Na pasta services ficam o código e os testes de cada serviço, junto com o Dockerfile, que explica como construir sua imagem. Já o arquivo docker-compose.yml reúne os quatro serviços da loja, o banco PostgreSQL, o Redis e as ferramentas de acompanhamento Jaeger, Prometheus e Grafana. Com o Docker aberto, os comandos abaixo podem ser executados na mesma pasta do README.

- docker compose up -d --build --wait
- docker compose ps
- node scripts/smoke.js

O primeiro comando prepara as imagens e inicia o ambiente, enquanto o segundo permite conferir quais containers estão em funcionamento. Com tudo pronto, o terceiro usa o Node.js para criar um pedido e procurá-lo na lista da loja. O painel fica disponível em http://127.0.0.1:8080 e, ao terminar, é possível encerrar o ambiente com docker compose down. Os dados permanecem nos volumes para a próxima execução.

Já o Compose organiza a rede de entrada e a comunicação interna para que um serviço encontre o outro pelo nome. As configurações de desenvolvimento estão no arquivo .env.example. A construção das imagens também foi dividida em duas etapas, de modo que o código e os testes são verificados antes de copiar os arquivos necessários para a execução. Com a imagem pronta, a aplicação funciona com o usuário 10001, sem precisar de permissão do administrador.

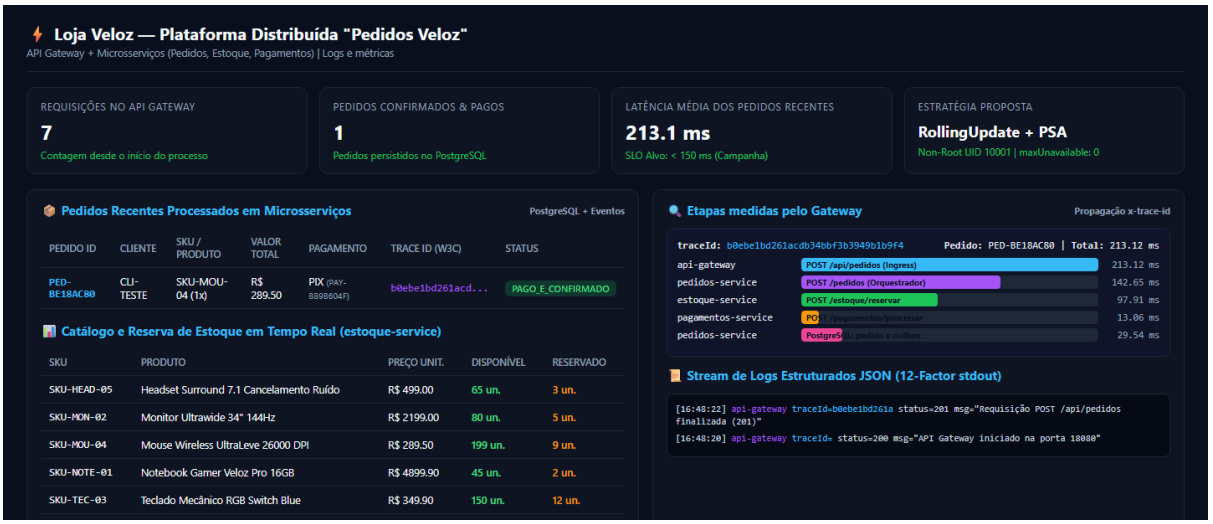
RESULTADO NO DOCKER COMPOSE

Na execução de 11 de novembro, os nove containers estavam em funcionamento. Seis exibiam a indicação “healthy”, que confirma a verificação de saúde, e os outros três apareciam como ativos. Ao comprar um mouse, o teste recebeu a resposta HTTP 201 e o número PED-475B842A. A quantidade disponível passou de 200 para 199 e o aviso de criação do pedido apareceu tanto no registro de publicação do banco quanto no Redis, como mostram os arquivos em evidencias/compose.

O PEDIDO DO CLIENTE NO WINDOWS

No teste, a compra usada como exemplo foi a de um mouse SKU-MOU-04 por R$

289,50, com pagamento simulado por PIX. Depois que a aplicação devolveu o número PED-BE18AC80, a consulta encontrou esse mesmo pedido na lista, como aparece no painel abaixo.

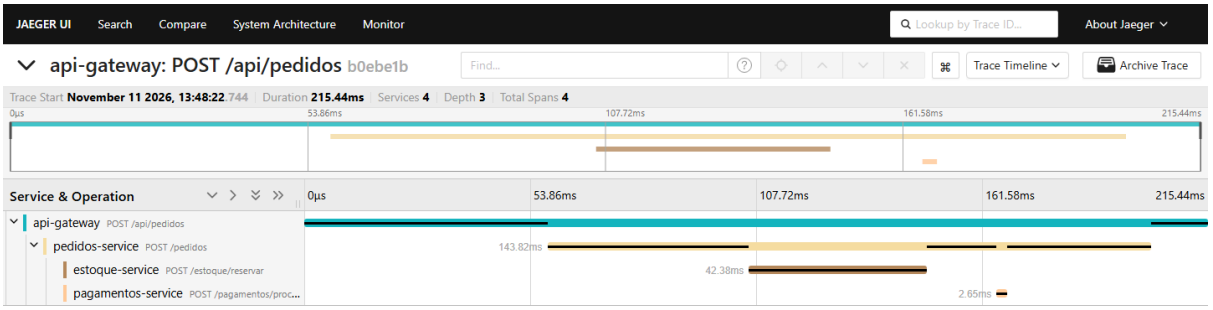


Com esse pedido, também foi possível acompanhar o que acontece quando o aviso de uma compra precisa esperar para ser enviado. O evento EVT-BBA2F023 já estava no banco como pendente quando o Memurai, compatível com o Redis no Windows, foi iniciado. Em seguida, o mesmo identificador apareceu na lista de mensagens do Redis e o banco passou a indicar que a publicação havia sido concluída.

Em outro teste, a compra de 11 notebooks ultrapassou o limite do simulador e recebeu a resposta HTTP 422. Além de conferir a recusa, foi preciso consultar o estoque para saber se a reserva havia sido desfeita. Os 45 notebooks continuavam disponíveis e a quantidade reservada voltou a dois, o mesmo valor que aparecia antes da tentativa.

COMO ENCONTRAR UMA FALHA

Para acompanhar o pedido do painel anterior, o Gateway cria um identificador e o envia aos demais serviços. Cada um registra quanto tempo gastou na sua parte da compra, e o Jaeger reúne essas informações em uma única tela. Na figura abaixo, cada barra indica quando uma etapa começou e quanto tempo levou.



Durante uma campanha, a equipe pode usar as métricas coletadas pelo Prometheus e consultadas no Grafana para acompanhar quantas compras falham e quanto tempo os clientes esperam. Também vale observar há quanto tempo o aviso mais antigo aguarda envio, pois a loja pode continuar vendendo enquanto as mensagens se acumulam. Quando uma compra exige uma análise mais detalhada, seu identificador permite encontrar nos logs o serviço que atendeu a chamada e o resultado que ele devolveu.

PUBLICAÇÃO DAS IMAGENS

O arquivo ci-cd.yaml descreve o caminho de uma mudança até a publicação das imagens no GitHub. Antes de liberar uma versão, o GitHub Actions verifica o código, os Dockerfiles e as configurações, executa os testes e faz uma compra com o Compose. O registro que acompanha o projeto mostra 13 testes aprovados. Depois destas verificações, o Trivy examina as imagens em busca de vulnerabilidades antes do envio ao GHCR, o registro de imagens do GitHub.

Cada imagem leva a versão e o código da alteração no Git, o que permite saber qual revisão foi publicada. As credenciais ficam nos secrets do GitHub. Na execução nº 2, referente ao commit 32f28a4, os quatro serviços passaram pelas verificações e suas imagens foram publicadas no GHCR em um processo que levou 3 min 22 s.

ATUALIZAÇÃO E SEGURANÇA NO KUBERNETES

No laboratório de 12 de novembro, os quatro serviços da aplicação foram distribuídos em nove pods, as unidades que o Kubernetes do Docker Desktop coloca em execução. O banco, o Redis e o Jaeger continuaram acessíveis pela rede

do laboratório. Ao enviar uma compra pelo Gateway do cluster, o teste recebeu HTTP 201 e o número PED-90C870D6. As configurações estão na pasta k8s e os registros da compra em evidencias/kubernetes.

No Kubernetes, os Deployments definem como as cópias de cada serviço são mantidas e substituídas, enquanto os Services oferecem um endereço que continua o mesmo quando uma cópia muda. As portas e os endereços ficam no ConfigMap, e as credenciais, como a senha do banco, são fornecidas por um Secret. Para saber quais cópias já podem receber pedidos, o Kubernetes verifica periodicamente a rota /ready. A rota /health indica se o processo continua respondendo e ajuda a identificar quando é preciso reiniciá-lo.

Como nenhum dos quatro serviços precisa de permissão de administrador, escolhi o perfil restricted do Pod Security Admission para o ambiente da loja, chamado pedidos-veloz. Com essa configuração, o próprio Kubernetes verifica as regras de segurança antes de aceitar um novo pod. No teste, ele recusou um pod que não atendia aos requisitos e aceitou os nove pods da aplicação sem emitir avisos. Também configurei os arquivos da aplicação como somente leitura e limitei as conexões internas com regras de rede, para que cada serviço receba chamadas apenas de quem precisa acessá-lo.

A primeira atualização mostrou uma falha que aparecia enquanto o sistema atendia clientes. Das 6.046 consultas, três receberam erro 502 porque ainda chegavam chamadas à versão que estava encerrando. Na versão 1.2.1, o serviço passou a avisar que deixaria de receber tráfego e a esperar cinco segundos antes de parar. O Gateway também passou a repetir uma leitura quando a conexão falha. Com esses ajustes, as 25.979 consultas do novo ensaio receberam resposta 200.

ESCALA DURANTE O PICO

Quando o movimento aumenta, o HPA ajusta o número de cópias de acordo com os recursos em uso. O Gateway pode passar de duas a dez cópias, com alvo de 70% da CPU solicitada, enquanto Pedidos pode variar de três a doze e leva em conta também a memória. Estoque e Pagamentos mantêm duas cópias cada e contam com uma configuração opcional que sugere ajustes nos recursos reservados.

Durante quatro minutos, o teste fez 110.139 consultas à lista de pedidos e todas receberam resposta 200. Nesse período, o Gateway passou de duas para dez cópias e Pedidos passou de três para dez. Para estimar o movimento de uma campanha real, ainda seria necessário testar o que acontece quando vários clientes tentam comprar as últimas unidades de um mesmo produto, pois esse ensaio mediu a consulta à lista em um computador de teste.

INFRAESTRUTURA E RESULTADOS

O Terraform reúne no repositório a descrição da rede, do cluster Kubernetes e do banco PostgreSQL previstos para a AWS. Dessa forma, a equipe consegue revisar as mudanças antes de aplicá-las e aproveitar o mesmo modelo para ambientes de tamanhos diferentes.

QUADRO DO PROJETO

Dificuldade	Ação	Resultado
Interrupção na atualização	Aguardar o encerramento das chamadas em andamento.	A repetição do ensaio teve 25.979 respostas 200.
Aumento dos acessos	Ajustar as réplicas de Gateway e Pedidos pelo uso de recursos.	Ambos chegaram a dez réplicas no teste de consultas.
Dificuldade para localizar falhas	Acompanhar o mesmo pedido nos quatro serviços.	A consulta no Jaeger reuniu as etapas da compra.

CONCLUSÃO

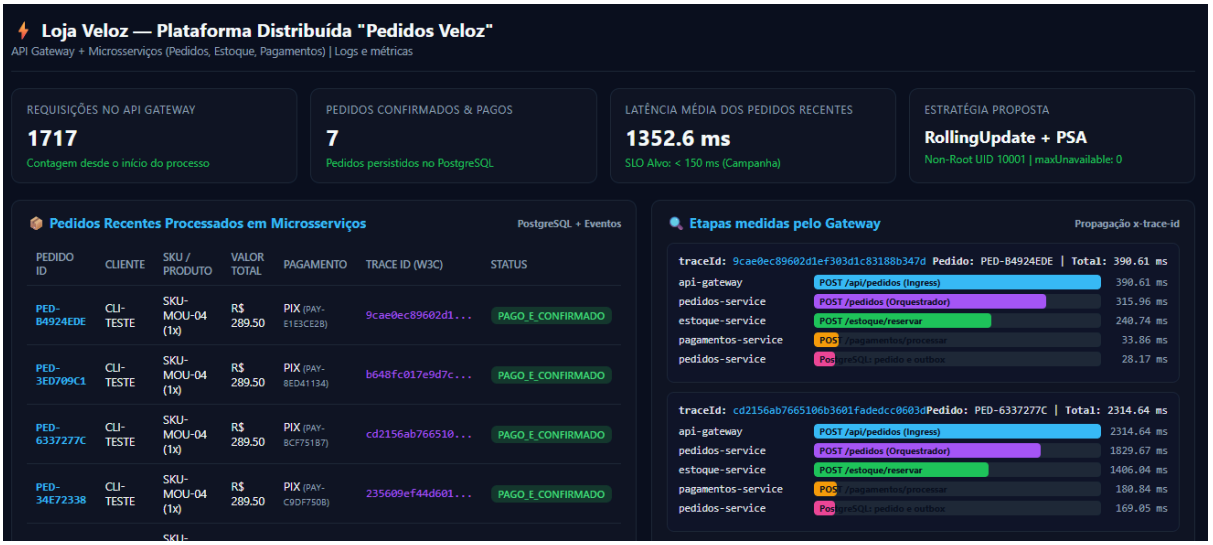
Ao acompanhar a compra do mouse, foi possível conferir se o pedido aparecia na lista, se a quantidade disponível diminuía e se o aviso chegava aos outros serviços. Já a recusa dos notebooks mostrou por que vale consultar o estoque mesmo quando o cliente recebe uma mensagem de erro. O teste de atualização revelou outro cuidado para a campanha: antes de desligar uma versão, a loja precisa dar tempo para que ela termine de atender quem já está comprando. Foi essa mudança que permitiu repetir o ensaio sem perder nenhuma das consultas.

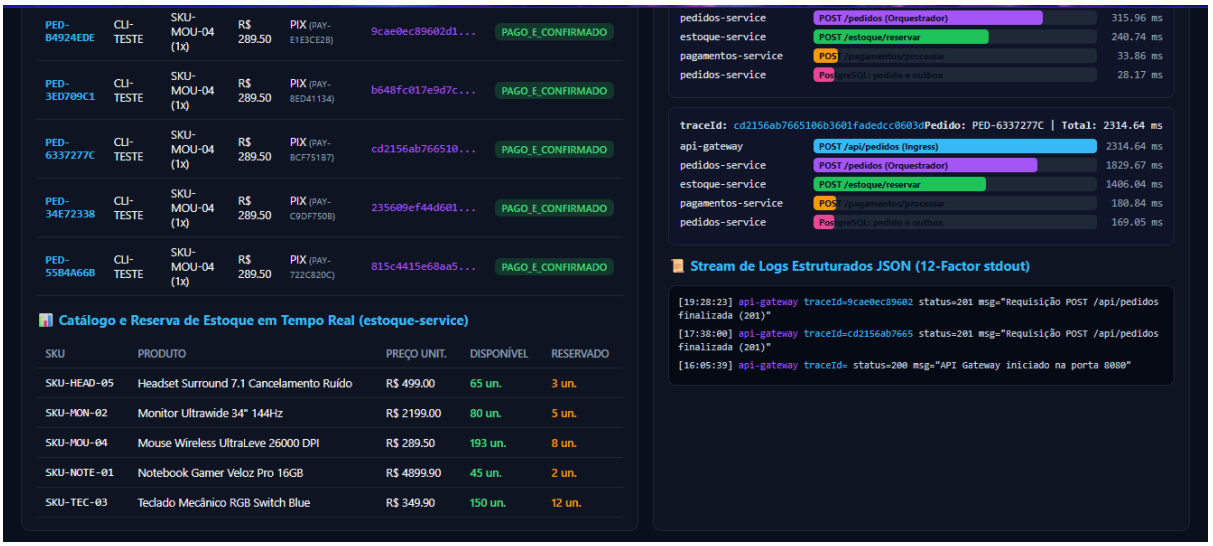
LINKS:

Vídeo: https://youtu.be/XociYp9OuyI

Repositório do projeto: https://github.com/barbaraleao-28/loja-veloz-devops

PRINTS VIDEO:





JAEGER UI | Procurar | Comparar | Arquitetura do sistema | Monitor | Pesquisa por ID de rastr

Procurar | Carregar

Serviço (0)

-

Operação (0)

todos

Etiquetas

http.status_code=200 erro=true

Retrospectiva

Última hora

Duração máxima | Duração mínima

Exemplo: 1,2s, ... | Exemplo: 1,2s, ...

Limitar resultados



Prometheus | Alerts | Graph | Status | Help

Targets

All scrape pools | All | Unhealthy | Collapse All | Filter by endpoint or labels | Unknown | Unhealthy | Healthy

api-gateway (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://api-gateway:8080/metrics	UP	instance="api-gateway:8080" job="api-gateway" service="api-gateway" tier="frontend"	4.877s ago	10.664ms	

estoque-service (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://estoque-service:3003/metrics	UP	instance="estoque-service:3003" job="estoque-service" service="estoque-service" tier="backend"	6.798s ago	3.114ms	

pagamentos-service (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://pagamentos-service:3002/metrics	UP	instance="pagamentos-service:3002" job="pagamentos-service" service="pagamentos-service" tier="backend"	3.374s ago	14.735ms	

pedidos-service (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://pedidos-service:3001/metrics	UP	instance="pedidos-service:3001" job="pedidos-service" service="pedidos-service" tier="backend"	14.677s ago	6.518ms	

```
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz> docker compose ps
NAME                      IMAGE                                        COMMAND                  SERVICE              CREATED        STATUS
        PORTS
veloz-api-gateway         ghcr.io/loja-veloz/api-gateway:1.2.1         "docker-entrypoint.s…"   api-gateway          16 hours ago   Up 3 hours (he
althy)   127.0.0.1:8080->8080/tcp
veloz-estoque-service     ghcr.io/loja-veloz/estoque-service:1.2.1     "docker-entrypoint.s…"   estoque-service      16 hours ago   Up 3 hours (he
althy)   127.0.0.1:3003->3003/tcp
veloz-grafana             grafana/grafana:11.0.0                       "/run.sh"                grafana              17 hours ago   Up 3 hours
        127.0.0.1:3000->3000/tcp
veloz-jaeger              jaegertracing/all-in-one:1.57                "/go/bin/all-in-one-…"   jaeger               17 hours ago   Up 3 hours
        127.0.0.1:4317-4318->4317-4318/tcp, 127.0.0.1:16686->16686/tcp
veloz-pagamentos-service  ghcr.io/loja-veloz/pagamentos-service:1.2.1  "docker-entrypoint.s…"   pagamentos-service   16 hours ago   Up 3 hours (he
althy)   127.0.0.1:3002->3002/tcp
veloz-pedidos-service     ghcr.io/loja-veloz/pedidos-service:1.2.1     "docker-entrypoint.s…"   pedidos-service      16 hours ago   Up 3 hours (he
althy)   127.0.0.1:3001->3001/tcp
veloz-postgres            postgres:16-alpine                           "docker-entrypoint.s…"   postgres             17 hours ago   Up 3 hours (he
althy)   127.0.0.1:5432->5432/tcp
veloz-prometheus          prom/prometheus:v2.52.0                      "/bin/prometheus --c…"   prometheus           17 hours ago   Up 3 hours
        127.0.0.1:9090->9090/tcp
veloz-redis-eventos       redis:7.2-alpine                             "docker-entrypoint.s…"   redis                17 hours ago   Up 3 hours (he
althy)   127.0.0.1:6379->6379/tcp
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz> node scripts/smoke.js
{
  "http": 201,
  "pedido": {
    "pedido_id": "PED-B4924EDE",
    "cliente_id": "CLI-TESTE",
    "sku": "SKU-MOU-04",
    "produto_nome": "Mouse Wireless UltraLeve 26000 DPI",
    "quantidade": 1,
    "valor_total": 289.5,
```

```
    "sku": "SKU-MOU-04",
    "produto_nome": "Mouse Wireless UltraLeve 26000 DPI",
    "quantidade": 1,
    "valor_total": 289.5,
    "metodo_pagamento": "PIX",
    "reserva_id": "RES-798825a7-eb3a-4cb3-ad1a-6617a3b671fb",
    "pagamento_id": "PAY-E1E3CE2B",
    "autorizacao_externa": "AUTH-9C27072AFAB1",
    "status": "PAGO_E_CONFIRMADO",
    "trace_id": "9cae0ec89602d1ef303d1c83188b347d",
    "criado_em": "2026-11-12T19:28:23.627Z"
  },
  "traceId": "9cae0ec89602d1ef303d1c83188b347d",
  "consulta": "OK"
}
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz> .\scripts\testar-hpa.ps1 -DuracaoS 120 -Concorrencia 60
Carga iniciada por 120 s com 60 conexões. Acompanhando o HPA a cada 15 s...
--- 16:28:39
api-gateway-hpa       Deployment/api-gateway       cpu: 1%/70%                  2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 2%/70%, memory: 12%/80%   3     12    3     16h
--- 16:28:54
api-gateway-hpa       Deployment/api-gateway       cpu: 1%/70%                  2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 4%/70%, memory: 12%/80%   3     12    3     16h
--- 16:29:09
api-gateway-hpa       Deployment/api-gateway       cpu: 15%/70%                 2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 44%/70%, memory: 13%/80%   3     12    3     16h
--- 16:29:25
api-gateway-hpa       Deployment/api-gateway       cpu: 159%/70%                2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 151%/70%, memory: 13%/80%   3     12    3     16h
--- 16:29:41
api-gateway-hpa       Deployment/api-gateway       cpu: 331%/70%                2     10    4     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 171%/70%, memory: 14%/80%   3     12    6     16h
--- 16:29:56
api-gateway-hpa       Deployment/api-gateway       cpu: 317%/70%                2     10    8     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 119%/70%, memory: 15%/80%   3     12    6     16h
```

```
api-gateway-hpa       Deployment/api-gateway       cpu: 15%/70%                 2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 44%/70%, memory: 13%/80%   3     12    3     16h
--- 16:29:25
api-gateway-hpa       Deployment/api-gateway       cpu: 159%/70%                2     10    2     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 151%/70%, memory: 13%/80%   3     12    3     16h
--- 16:29:41
api-gateway-hpa       Deployment/api-gateway       cpu: 331%/70%                2     10    4     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 171%/70%, memory: 14%/80%   3     12    6     16h
--- 16:29:56
api-gateway-hpa       Deployment/api-gateway       cpu: 317%/70%                2     10    8     16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 119%/70%, memory: 15%/80%   3     12    6     16h
--- 16:30:21
api-gateway-hpa       Deployment/api-gateway       cpu: 88%/70%                 2     10    10    16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 103%/70%, memory: 13%/80%   3     12    6     16h
--- 16:30:38
api-gateway-hpa       Deployment/api-gateway       cpu: 88%/70%                 2     10    10    16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 68%/70%, memory: 13%/80%   3     12    9     16h
--- 16:30:55
api-gateway-hpa       Deployment/api-gateway       cpu: 79%/70%                 2     10    10    16h
pedidos-service-hpa   Deployment/pedidos-service   cpu: 67%/70%, memory: 14%/80%   3     12    9     16h
--- Resultado da carga
{"total":14127,"por_status":{"200":14015,"503":6,"erro_rede":106},"sucesso_pct":99.21}
--- Eventos de escala
Após ~5 min sem carga o HPA volta ao mínimo (stabilizationWindowSeconds: 300).
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz>
```

```
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz> kubectl -n pedidos-veloz get hpa
NAME                 REFERENCE                    TARGETS                        MINPODS   MAXPODS   REPLICAS   AGE
api-gateway-hpa      Deployment/api-gateway       cpu: 0%/70%                    2         10        2          16h
pedidos-service-hpa  Deployment/pedidos-service   cpu: 2%/70%, memory: 12%/80%   3         12        3          16h
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz> kubectl -n pedidos-veloz get hpa
NAME                 REFERENCE                    TARGETS                          MINPODS   MAXPODS   REPLICAS   AGE
api-gateway-hpa      Deployment/api-gateway       cpu: 331%/70%                    2         10        4          16h
pedidos-service-hpa  Deployment/pedidos-service   cpu: 171%/70%, memory: 14%/80%   3         12        6          16h
PS C:\Users\Dell\OneDrive\Área de Trabalho\EAD_Devops\EAD-Loja-Veloz>
```